

04 - Classification

Class Imbalance, Feature Selection and Non Binary Classification

Alessandro Bregoli

2025

Introduction

In this **tutorial** we will **introduce** the following concept:

- **Classification Performance**
 - Confusion Matrix
 - Precision
 - Recall
 - Accuracy
 - Cumulative Gain
 - Lift Chart
 - ROC Curve
 - AUC-ROC
- **Class Imbalance Problem**
 - Cost Sensitive Classifier
 - Undersampling
- **Feature Selection**
 - Univariate
 - Multivariate
 - Wrapper
- **Non Binary Classification**



Classification Performance Metrics

The limitation of accuracy

Accuracy is a very common metric in machine learning used to **quickly** access the **overall performance** of the developed **model**.



The limitation of accuracy

Accuracy is a very common metric in machine learning used to **quickly** access the **overall performance** of the developed **model**.

However, **accuracy** treats **every class** as **equally important**.



The limitation of accuracy

Accuracy is a very common metric in machine learning used to **quickly** access the **overall performance** of the developed **model**.

However, **accuracy** treats **every class** as **equally important**.

This can be a **strong limitation** especially in situation of **imbalance** **classes** where the **rare class** is the **class of interest**.



The limitation of accuracy

Accuracy is a very common metric in machine learning used to **quickly** access the **overall performance** of the developed **model**.

However, **accuracy** treats **every class** as **equally important**.

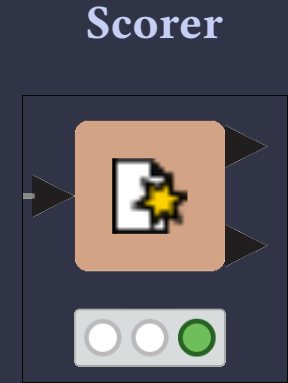
This can be a **strong limitation** especially in situation of **imbalance** **classes** where the **rare class** is the **class of interest**.

Furthermore, the **accuracy** does **not** allow to **distinguish** among the different **errors** that a model can commit.



Scorer Node - Confusion Matrix

The *validation* node compute the **confusion matrix** (fisrt output).



Scorer Node - Confusion Matrix

The *validation* node compute the **confusion matrix** (fisrt output).

		Predicted Class	
		False	True
Actual Class	False	TN	FP
	True	FN	TP

- TP: True Positive
- TN: True Negative
- FP: False Positive
- FN: False Negative

Scorer



Scorer Node - Confusion Matrix

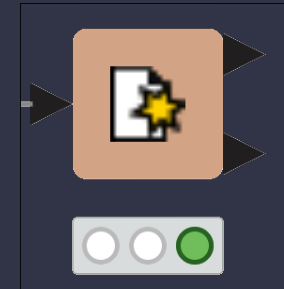
The *validation* node compute the **confusion matrix** (fisrt output).

	Predicted Class		
	False	True	
	False	TN	FP
Actual Class	True	FN	TP

- TP: True Positive
- TN: True Negative
- FP: False Positive
- FN: False Negative

The **confusion matrix** is useful both to **summarize** the **performance** of the algorithm and as a **starting point** for many different **score functions**.

Scorer



Scorer Node - Statistics

Starting from the **confusion matrix**, the *validation* node compute a set of validation metrics (second output) such as:

-
-
-
-

Scorer



Scorer Node - Statistics

Starting from the **confusion matrix**, the *validation* node compute a set of validation metrics (second output) such as:

- **Accuracy**

-
-
-

Accuracy measures the **proportion** of cases **correctly classified**.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Scorer



Scorer Node - Statistics

Starting from the **confusion matrix**, the *validation* node compute a set of validation metrics (second output) such as:

- Accuracy
- **Precision**
-
-

Precision measures the proportion of **positive predictions** that were **actually correct**.

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

Scorer



Scorer Node - Statistics

Starting from the **confusion matrix**, the *validation* node compute a set of validation metrics (second output) such as:

- Accuracy
- Precision
- **Recall**
-

Recall measures the **proportion of actual positive cases** that were **correctly identified**.

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

Scorer



Scorer Node - Statistics

Starting from the **confusion matrix**, the *validation* node compute a set of validation metrics (second output) such as:

- Accuracy
- Precision
- Recall
- **F1**

F1 is the **harmonic mean** of **precision** and **recall**.

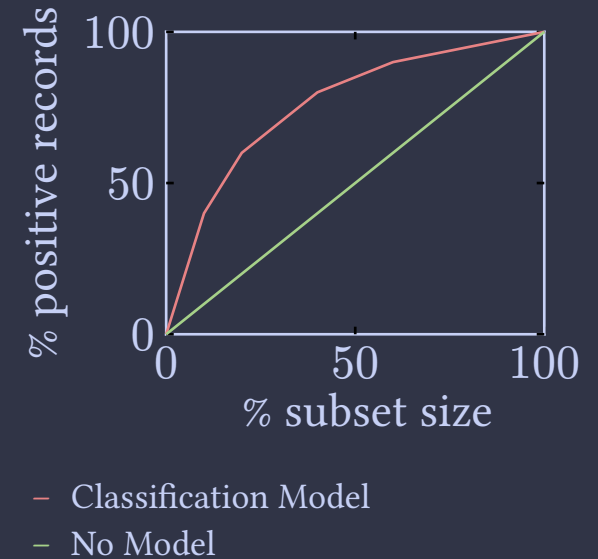
$$F1 = \frac{2}{\text{Precision}^{-1} + \text{Recall}^{-1}}$$

Scorer



Cumulative Gain

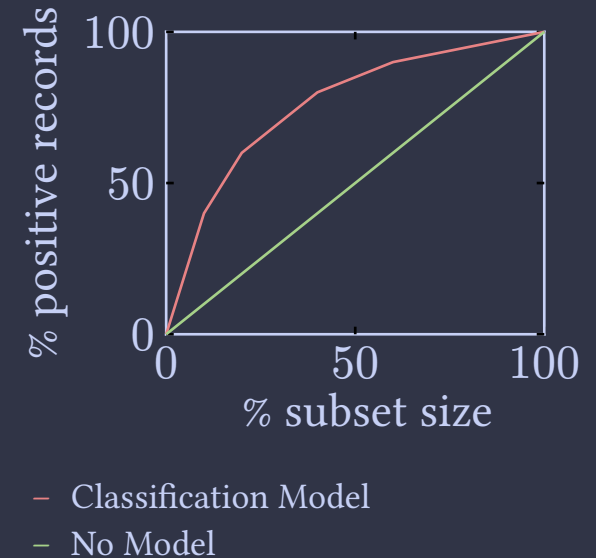
A **classifier** not only predicts the **most likely class** for an input but also provides a **probability score** across all **possible classes**.



Cumulative Gain

A **classifier** not only predicts the **most likely class** for an input but also provides a **probability score** across all **possible classes**.

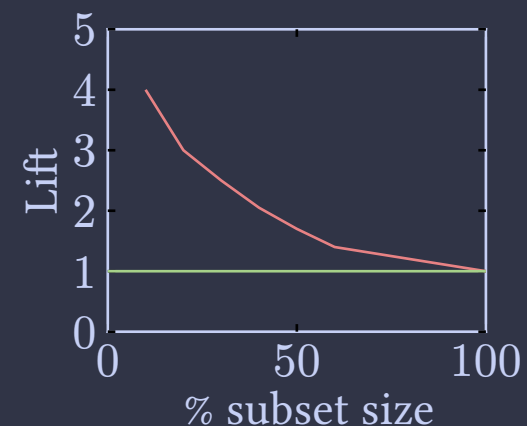
Thus, we can **rank the records** according to the model's **predicted probability** of a positive class and observe the **percentage** (over the entire dataset) of **true positive** records observed as we **move down the sorted list**.



Lift Factor

The **Cumulative Lift chart** is strictly related the Cumulative Gain:

$$\text{Lift} = \frac{\% \text{ positive records}}{\% \text{ subset size}}$$

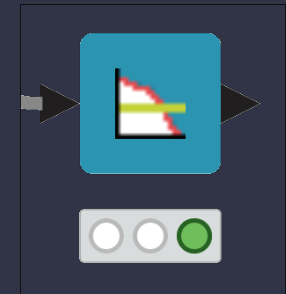


- Classification Model
- No Model

Lift Chart node

The *Lift Chart (Legacy)* node implements both the **Cumulative Lift Chart** and the **Cumulative Gain Chart**.

Lift Chart
(Legacy)



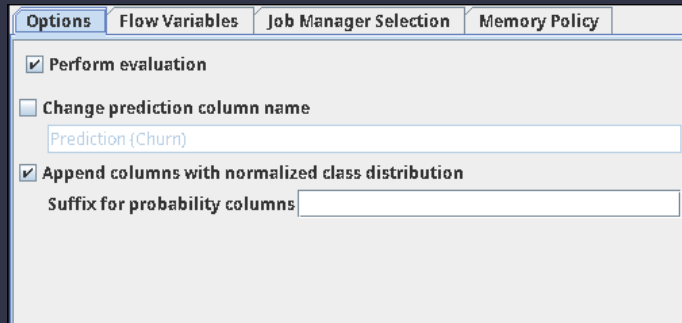
Weka Predictor



Lift Chart node

The *Lift Chart (Legacy)* node implements both the **Cumulative Lift Chart** and the **Cumulative Gain Chart**.

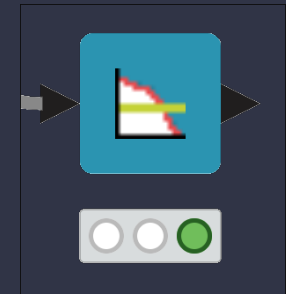
Predictor Config



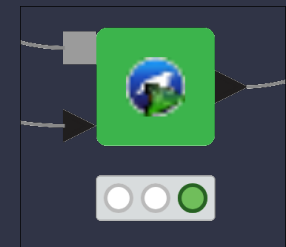
The image shows the 'Predictor Config' dialog box with four tabs: 'Options', 'Flow Variables', 'Job Manager Selection', and 'Memory Policy'. The 'Options' tab is active and contains the following settings:

- ☒ Perform evaluation
- ☐ Change prediction column name
Prediction (Churn)
- ☒ Append columns with normalized class distribution
Suffix for probability columns

Lift Chart (Legacy)



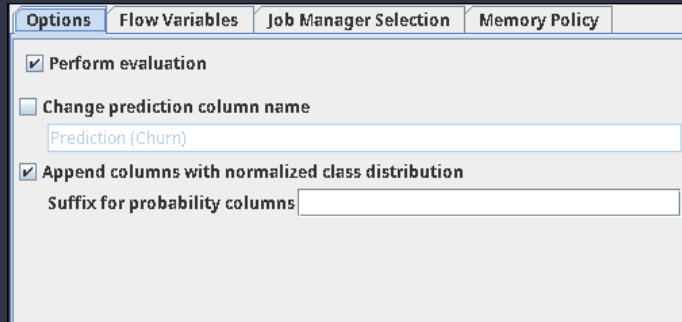
Weka Predictor



Lift Chart node

The *Lift Chart (Legacy)* node implements both the **Cumulative Lift Chart** and the **Cumulative Gain Chart**.

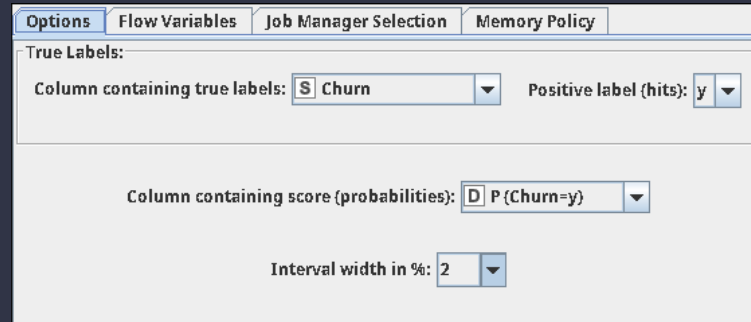
Predictor Config



The Predictor Config dialog box has four tabs: Options, Flow Variables, Job Manager Selection, and Memory Policy. The Options tab is active, showing the following settings:

- ☒ Perform evaluation
- ☐ Change prediction column name
Prediction (Churn)
- ☒ Append columns with normalized class distribution
Suffix for probability columns

Lift Chart Config



The Lift Chart Config dialog box has four tabs: Options, Flow Variables, Job Manager Selection, and Memory Policy. The Options tab is active, showing the following settings:

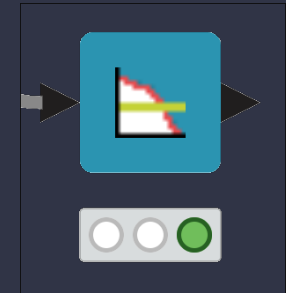
True Labels:

Column containing true labels: Positive label (hits):

Column containing score (probabilities):

Interval width in %:

Lift Chart (Legacy)

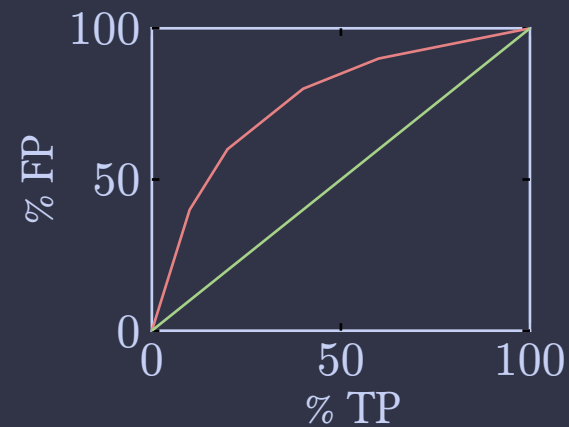


Weka Predictor



ROC Curve

The **Reviver Operating Characteristic** (ROC) curve is similar to the **Cumulative Gain**.

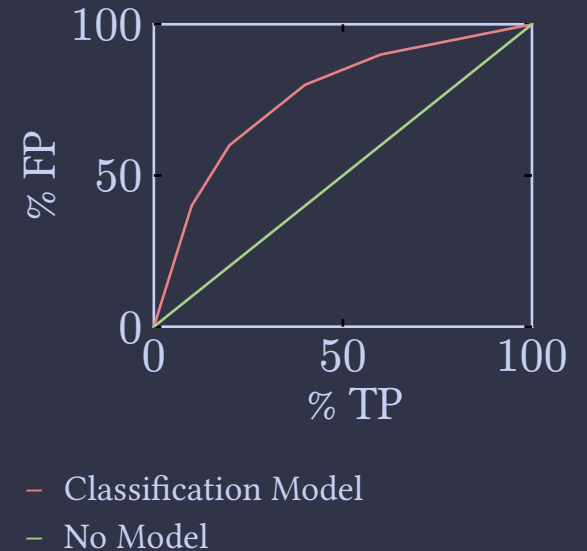


- Classification Model
- No Model

ROC Curve

The **Receiver Operating Characteristic** (ROC) curve is similar to the **Cumulative Gain**.

We **rank the records** according to the model's **predicted probability** of a positive class and observe the **percentage of True Positive** and **False Positive** as we **move down the sorted list**.

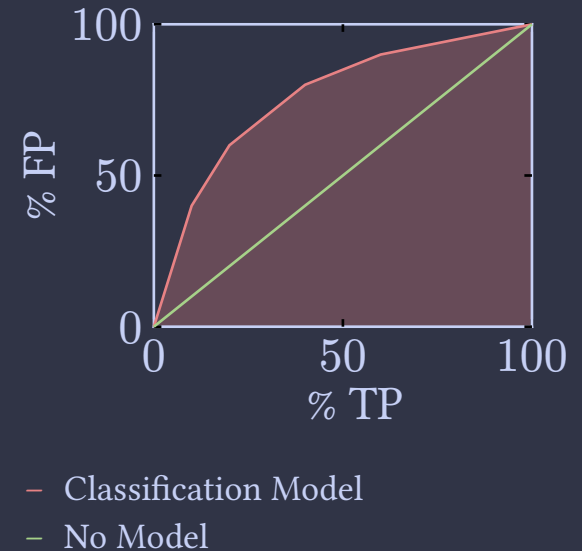


ROC Curve

The **Receiver Operating Characteristic** (ROC) curve is similar to the **Cumulative Gain**.

We **rank the records** according to the model's **predicted probability** of a positive class and observe the **percentage of True Positive** and **False Positive** as we **move down the sorted list**.

A common **single value metric** used to evaluate the performance of a classifier is the **Area under the ROC** (AUC) curve.



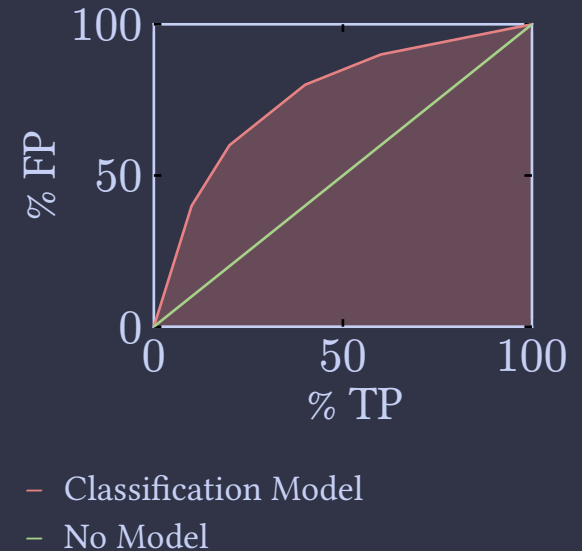
ROC Curve

The **Receiver Operating Characteristic** (ROC) curve is similar to the **Cumulative Gain**.

We **rank the records** according to the model's **predicted probability** of a positive class and observe the **percentage of True Positive** and **False Positive** as we **move down the sorted list**.

A common **single value metric** used to evaluate the performance of a classifier is the **Area under the ROC** (AUC) curve.

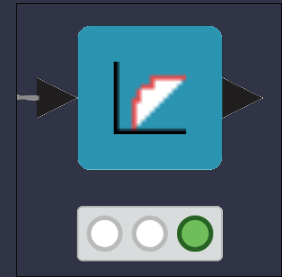
The **AUC** spans from **0.5** (no classifier) to **1.0** (perfect classifier).



Roc Curve Node

The *ROC Curve (Legacy)* node implements the ROC curve.

**Roc Curve
(Legacy)**

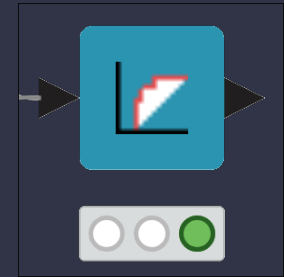


Roc Curve Node

The *ROC Curve (Legacy)* node implements the ROC curve.

- This node allows to view the **ROC curve** of a **model** compared to the **no model** curve.
- The **output** port returns **AUC score**.

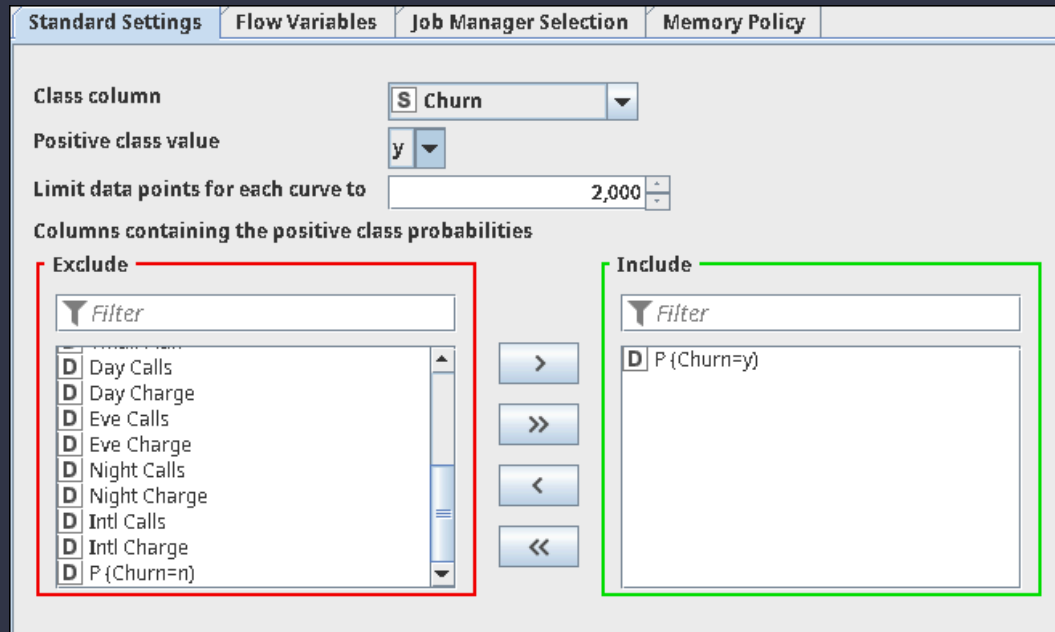
Roc Curve (Legacy)



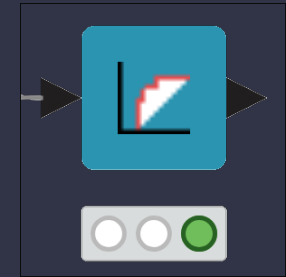
Roc Curve Node

The *ROC Curve (Legacy)* node implements the ROC curve.

- This node allows to view the **ROC curve** of a **model** compared to the **no model** curve.
- The **output** port returns **AUC score**.



Roc Curve (Legacy)



Hands on (20 minutes)

- **Load and prepare** the **Churn_yn.xls** dataset
- **Partition** the dataset (70% train - 30% test) with the **stratified** strategy using as seed **2025**.
- **Train** the following models:
 - **Logistic Regression** (weka)
 - Set ride param to 0.0
 - **Naive Bayes** (Knime standard library)
- **Compare** the performance of the two algorithms using:
 - Scorer
 - Lift Chart
 - ROC Curve
- Do all metrics agree on which is the best classifier?



Class Imbalance Problem

Introduction

The **class imbalance problem** occurs when a dataset has a **disproportion** in the number of record assigned to different **classes**.



Introduction

The **class imbalance problem** occurs when a dataset has a **disproportion** in the number of record assigned to different **classes**.

The class imbalance problem can introduce the following complication in the classification task:



Introduction

The **class imbalance problem** occurs when a dataset has a **disproportion** in the number of record assigned to different **classes**.

The class imbalance problem can introduce the following complication in the classification task:

- **Misleading Evaluation Metrics:** As seen in the previous section, some metrics, including accuracy, suffer from the presence of imbalance.
-



Introduction

The **class imbalance problem** occurs when a dataset has a **disproportion** in the number of record assigned to different **classes**.

The class imbalance problem can introduce the following complication in the classification task:

- **Misleading Evaluation Metrics:** As seen in the previous section, some metrics, including accuracy, suffer from the presence of imbalance.
- **Poor Performance:** the model struggle to generalize the properties of the minority class.



Cost Matrix

The **confusion matrix** can help us interpret the performance of a classifier even in presence of unbalance data.

Confusion Matrix

		Predicted Class	
		False	True
Actual Class	False	TN	FP
	True	FN	TP

Cost Matrix

The **confusion matrix** can help us interpret the performance of a classifier even in presence of unbalance data.

We can **exploit** the confusion matrix also for addressing the **training** of an unbalanced classifier.

Confusion Matrix

		Predicted Class	
		False	True
Actual Class	False	TN	FP
	True	FN	TP

Cost Matrix

The **confusion matrix** can help us interpret the performance of a classifier even in presence of unbalance data.

We can **exploit** the confusion matrix also for addressing the **training** of an unbalanced classifier.

The **Cost matrix** define the cost for making a particular prediction given the real class of the record.

$$\begin{aligned} \text{Cost} = & \text{TPC} \cdot \text{TP} \\ & + \text{TNC} \cdot \text{TN} \\ & + \text{FPC} \cdot \text{FP} \\ & + \text{FNC} \cdot \text{FN} \end{aligned}$$

Confusion Matrix

		Predicted Class	
		False	True
Actual Class	False	TN	FP
	True	FN	TP

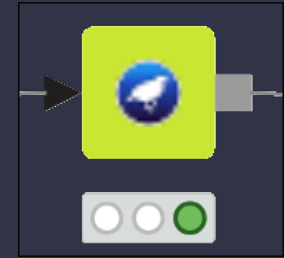
Cost Matrix

		Predicted Class	
		False	True
Actual Class	False	TNC	FPC
	True	FNC	TPC

Cost Sensitive Classifier

The *Cost Sensitive Classifier* node allows to **train** any **weka classifier** **optimizing** the **cost** defined with a cost matrix.

CostSensitiveClassifier

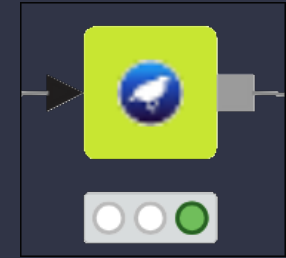


Cost Sensitive Classifier

The *Cost Sensitive Classifier* node allows to **train** any weka classifier **optimizing** the **cost** defined with a cost matrix.

CostSensitiveClassifier

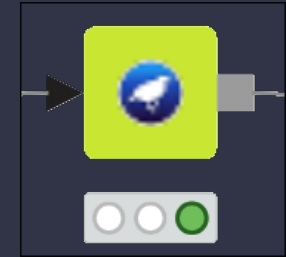
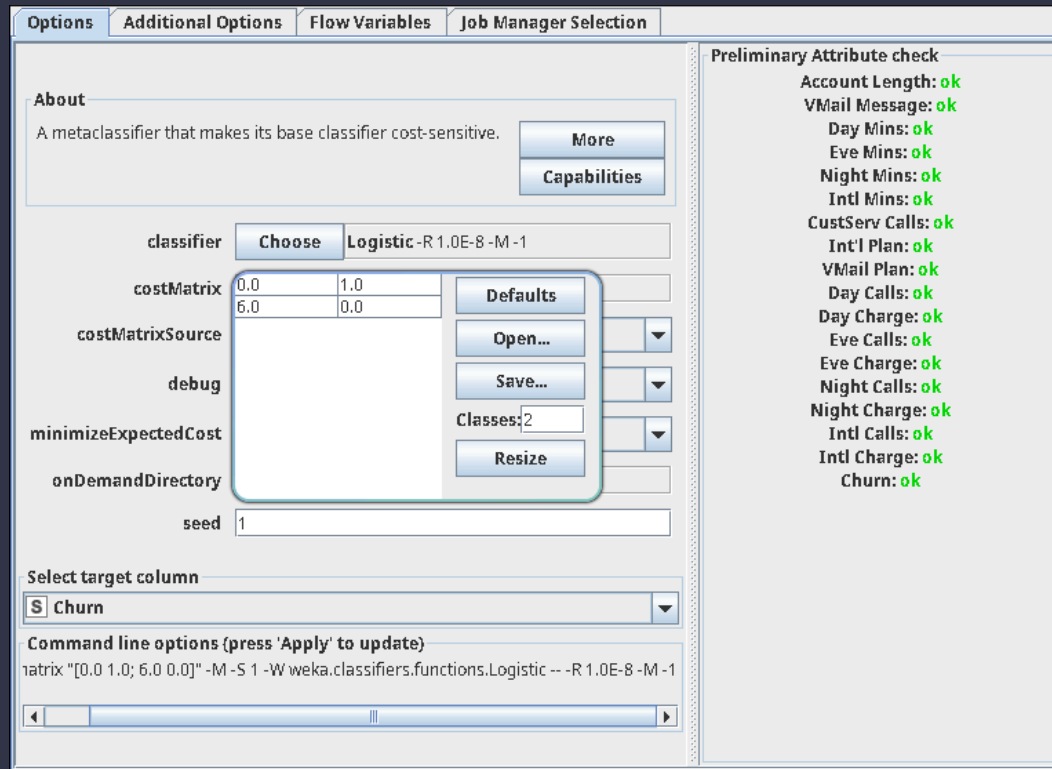
The screenshot shows the 'Options' tab of the 'Cost Sensitive Classifier' node. The 'About' section describes it as a metaclassifier that makes its base classifier cost-sensitive. The 'classifier' is set to 'Logistic -R 1.0E-8 -M -1'. The 'costMatrix' is '2 x 2 cost matrix'. The 'costMatrixSource' is 'Use explicit cost matrix'. The 'debug' option is 'False'. The 'minimizeExpectedCost' option is 'True'. The 'onDemandDirectory' is 'knime_5.8.0'. The 'seed' is '1'. The 'Select target column' dropdown is set to 'Churn'. The 'Command line options' section shows the command: `matrix "[0.0 1.0; 6.0 0.0]" -M -S 1 -W weka.classifiers.functions.Logistic -- -R 1.0E-8 -M -1`. The 'Preliminary Attribute check' section on the right lists various attributes with 'ok' status: Account Length, VMail Message, Day Mins, Eve Mins, Night Mins, Intl Mins, CustServ Calls, Intl Plan, VMail Plan, Day Calls, Day Charge, Eve Calls, Eve Charge, Night Calls, Night Charge, Intl Calls, Intl Charge, and Churn.



Cost Sensitive Classifier

The *Cost Sensitive Classifier* node allows to **train** any weka classifier **optimizing** the **cost** defined with a cost matrix.

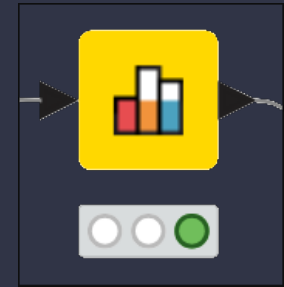
CostSensitiveClassifier



Undersampling

The *Equal Size Sampling* node, provided by Knime, allows to **undersample** the dataset such that the values in a **categorical column** are **equally distributed**.

Equal Size
Sampling

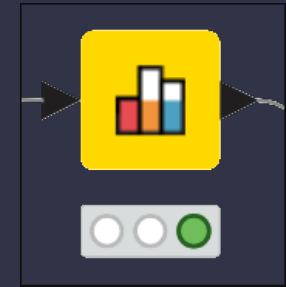


Undersampling

The *Equal Size Sampling* node, provided by Knime, allows to **undersample** the dataset such that the values in a **categorical column** are **equally distributed**.

By applying **undersampling** to the **target class**, we ensure that the **model** gives **equal importance** to learning from **every class**.

Equal Size Sampling



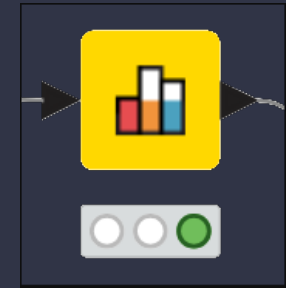
Undersampling

The *Equal Size Sampling* node, provided by Knime, allows to **undersample** the dataset such that the values in a **categorical column** are **equally distributed**.

By applying **undersampling** to the **target class**, we ensure that the **model** gives **equal importance** to learning from **every class**.

The main **drawback** of this approach is that we are using less data in the training phase.

Equal Size Sampling



Undersampling

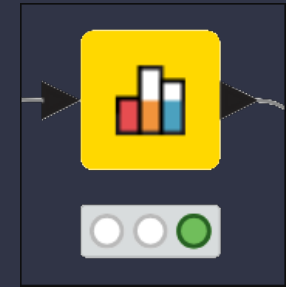
The *Equal Size Sampling* node, provided by Knime, allows to **undersample** the dataset such that the values in a **categorical column** are **equally distributed**.

By applying **undersampling** to the **target class**, we ensure that the **model** gives **equal importance** to learning from **every class**.

The main **drawback** of this approach is that we are using less data in the training phase.

The undersampling must be applied ONLY to the train set.

Equal Size
Sampling



Undersampling

The *Equal Size Sampling* node, provided by Knime, allows to **undersample** the dataset such that the values in a **categorical column** are **equally distributed**.

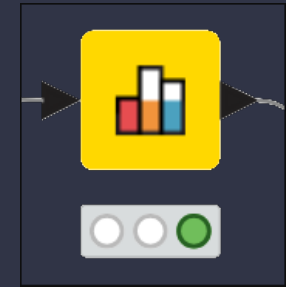
By applying **undersampling** to the **target class**, we ensure that the **model** gives **equal importance** to learning from **every class**.

The main **drawback** of this approach is that we are using less data in the training phase.

The undersampling must be applied **ONLY** to the train set.

Undersampling balances the **training set** to ensure uniform class focus. However, the **validation** must still be performed against a test set that reflects the **true class probabilities**.

Equal Size
Sampling



Hands on (20 minutes)

- **Load** and prepare the dataset **Churn_yn.xls**
- **Split** the dataset in **train (70%)** and **test (30%)** using **2025** as **seed**.
- **Train** and **validate** the following classifiers:
 - A Weka **Logistic Regression classifier** (*baseline*)
 - A Weka **Cost Sensitive Classifier** with the following parameters:
 - **Classifier**: Logistic Regression Classifier
 - **Cost Matrix**: TPC: 0, TNC: 0, FPC: 1, FNC: 6
 - A **Logistic Regression classifier** trained over an **Equal Size Sample** training set.
 - Use 2025 as seed for the Equal Size Sampling node.
- **Evaluate** the performance using a scorer node.
- **Compute** the **cost** for all the algorithms using the cost matrix:
 - TPC: 0, TNC: 0, FPC: 1, FNC: 6



Feature Selection

Introduction

Feature Selection is a procedure suitable for selecting the most **relevant variables**.



Introduction

Feature Selection is a procedure suitable for selecting the most **relevant variables**.

In the **classification context**, the goal is to **identify** those **features** that are responsible for an **effective predictions**.



Introduction

Feature Selection is a procedure suitable for selecting the most **relevant variables**.

In the **classification context**, the goal is to **identify** those **features** that are responsible for an **effective predictions**.

We can classify a feature selection algorithm with two approaches:



Introduction

Feature Selection is a procedure suitable for selecting the most **relevant variables**.

In the **classification context**, the goal is to **identify** those **features** that are responsible for an **effective predictions**.

We can classify a feature selection algorithm with two approaches:

1. Univariate/Multivariate

- **Univariate**: considers only one feature at time.
- **Multivariate**: considers also relations among features.

2. Filter/Wrapper

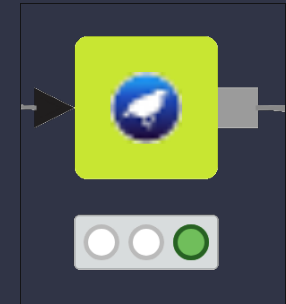
- **Filter**: selects the relevant feature before training the classification model.
- **Wrapper**: selects the features based on the performance of the classifier model.



AttributeSelectedClassifier Node

The *AttributeSelectedClassifier* is a node provided by weka implementing many different feature selection algorithms.

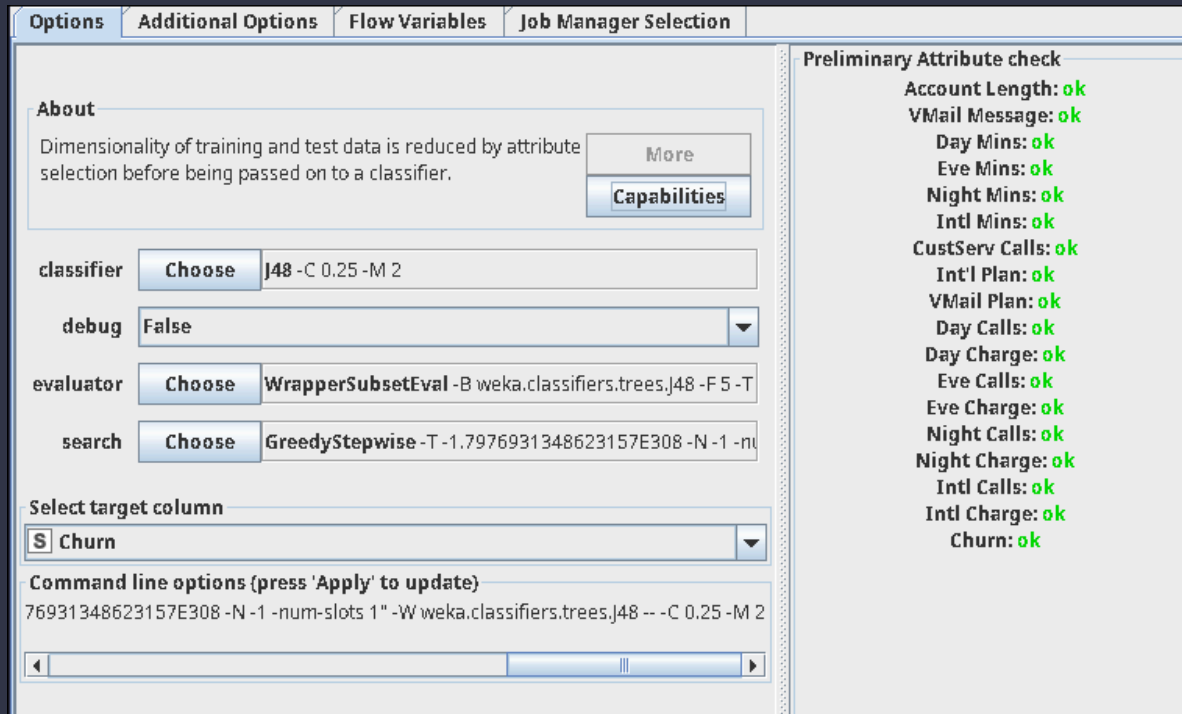
**Attribute
Selected
Classifier**



AttributeSelectedClassifier Node

The *AttributeSelectedClassifier* is a node provided by weka implementing many different feature selection algorithms.

Config



The screenshot shows the configuration window for the AttributeSelectedClassifier node. It has four tabs: Options, Additional Options, Flow Variables, and Job Manager Selection. The Options tab is active, showing an 'About' section with a 'More' button and a 'Capabilities' button. Below this are fields for 'classifier' (J48 -C 0.25 -M 2), 'debug' (False), 'evaluator' (WrapperSubsetEval -B weka.classifiers.trees.J48 -F 5 -T), and 'search' (GreedyStepwise -T -1.7976931348623157E308 -N -1 -nu). There is a 'Select target column' dropdown set to 'Churn'. At the bottom, there is a 'Command line options' section with the text '76931348623157E308 -N -1 -num-slots 1 -W weka.classifiers.trees.J48 -- -C 0.25 -M 2' and a scrollbar.

About

Dimensionality of training and test data is reduced by attribute selection before being passed on to a classifier. [More](#) [Capabilities](#)

classifier [Choose](#) J48 -C 0.25 -M 2

debug False

evaluator [Choose](#) WrapperSubsetEval -B weka.classifiers.trees.J48 -F 5 -T

search [Choose](#) GreedyStepwise -T -1.7976931348623157E308 -N -1 -nu

Select target column

S Churn

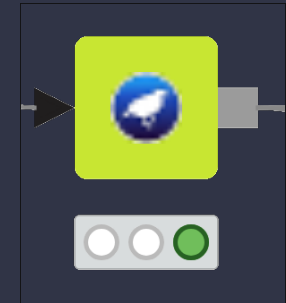
Command line options (press 'Apply' to update)

76931348623157E308 -N -1 -num-slots 1 -W weka.classifiers.trees.J48 -- -C 0.25 -M 2

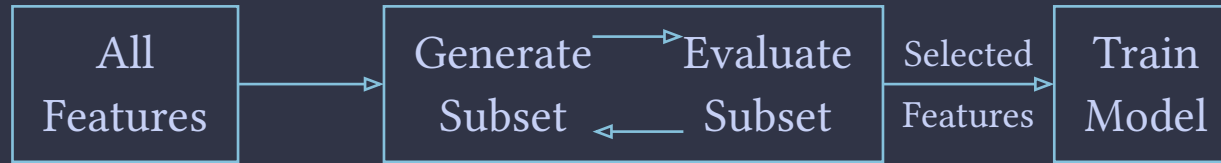
Preliminary Attribute check

- Account Length: **ok**
- VMail Message: **ok**
- Day Mins: **ok**
- Eve Mins: **ok**
- Night Mins: **ok**
- Intl Mins: **ok**
- CustServ Calls: **ok**
- Int'l Plan: **ok**
- VMail Plan: **ok**
- Day Calls: **ok**
- Day Charge: **ok**
- Eve Calls: **ok**
- Eve Charge: **ok**
- Night Calls: **ok**
- Night Charge: **ok**
- Intl Calls: **ok**
- Intl Charge: **ok**
- Churn: **ok**

Attribute Selected Classifier



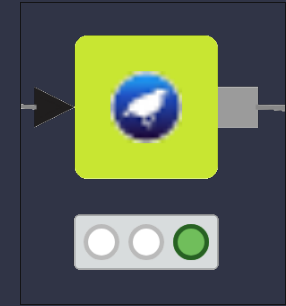
AttributeSelectedClassifier Node - Filter



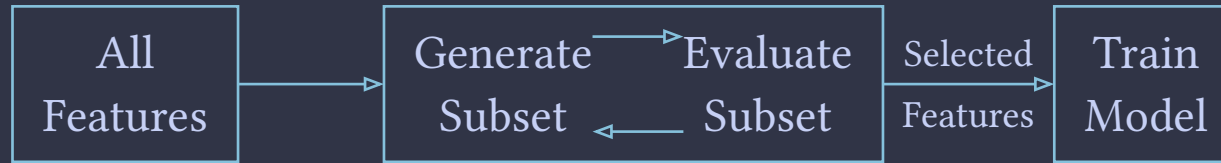
Univariate

Multivariate

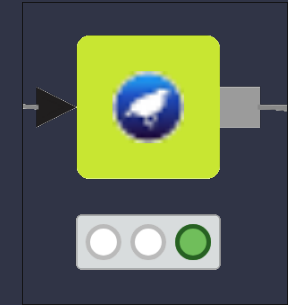
AttributeSelectedClassifier



AttributeSelectedClassifier Node - Filter



AttributeSelectedClassifier



Univariate

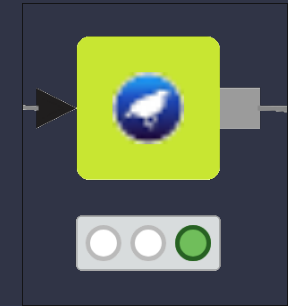
Multivariate

- **Evaluator:**
 - InfoGainAttributeEval
 - GainRatioAttributeEval
- **Search:**
 - Ranker
 - Config: number of features

AttributeSelectedClassifier Node - Filter



AttributeSelectedClassifier



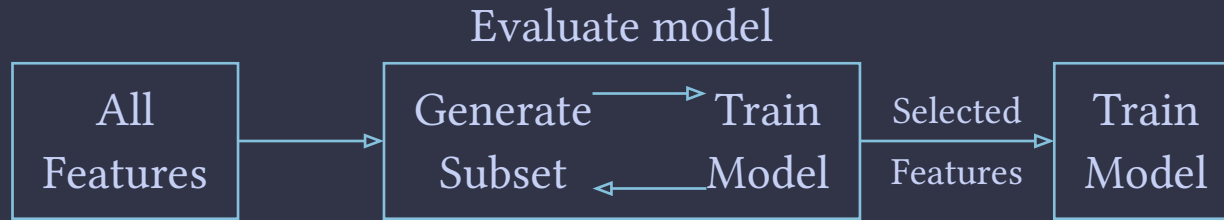
Univariate

- **Evaluator:**
 - InfoGainAttributeEval
 - GainRatioAttributeEval
- **Search:**
 - Ranker
 - Config: number of features

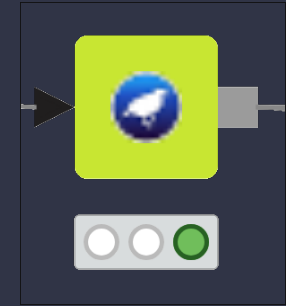
Multivariate

- **Evaluator:**
 - CfsSubsetEval
- **Search:**
 - BestFirst

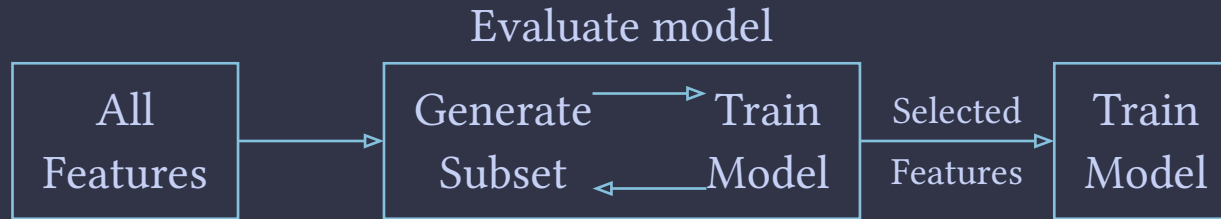
AttributeSelectedClassifier Node - Wrapper



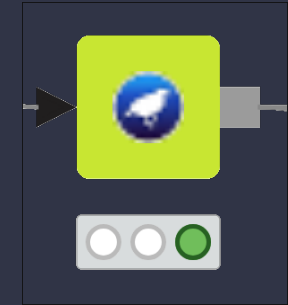
AttributeSelectedClassifier



AttributeSelectedClassifier Node - Wrapper



AttributeSelectedClassifier



- **Evaluator:**
 - WrapperSubsetEval
 - **Classifier:** any weka classifier
 - **evaluationMeasure:** Accuracy, AUC,...
 - **fold:** CV fold
- **Search:**
 - GreedyStepwise

AttributeSelectedClassifier Node - Output

```
AttributeSelectedClassifier:

=== Attribute Selection on all input data ===

Search Method:
    Attribute ranking.

Attribute Evaluator (supervised, Class {nominal}): 18 Churn:
    Information Gain Ranking Filter

Ranked attributes:
0.07537  3 DayMins
0.07537  11 DayCharge
0.043    8 IntlPlan
0.0395   7 CustServCalls
0.0091  17 IntlCharge

Selected attributes: 3,11,8,7,17 : 5

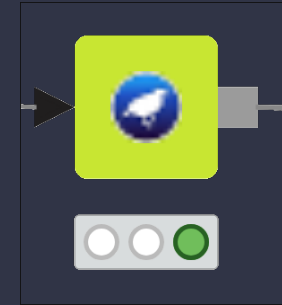
Header of reduced data:
@relation 'Weka-Instances-weka.filters.unsupervised.attribute.Remove-V-R3,11,8,7,17-18'

@attribute DayMins numeric
@attribute DayCharge numeric
@attribute IntlPlan numeric
@attribute CustServCalls numeric
@attribute IntlCharge numeric
@attribute Churn {n,y}

@data

Classifier Model
J48 pruned tree
-----
```

AttributeSelectedClassifier



Hands on (30 minutes)

- **Load** and prepare the dataset **Churn_yn.xls**
- **Split** the dataset in **train (70%)** and **test (30%)** using **2025** as **seed**.
- **Train** and **validate** an **AttributeSelectedClassifier** Node with a **J48** and the following configs:
 - (univariate) - **evaluator**: InfoGainAttributeEval - **search**: Ranker
 - (multivariate) - **evaluator**: CfsSubsetEval - **search**: BestFirst
 - (wrapper) **evaluator**: WrapperSubsetEval - **search**: GreedyStepwise
 - *hint: use the J48 model also inside the config of WrapperSubsetEval*
- Evaluate the performances of each model using a ROC curve.
- **Which approach works best?**
- **What happens if we change the estimatorMeasure in the WrapperSubsetEval configuration?**



Non Binary Classification

Multi-Class Classifiers

The **Churn_yn.xlsx** dataset used until now, is a binary classification dataset.



Multi-Class Classifiers

The **Churn_yn.xlsx** dataset used until now, is a binary classification dataset.

In this section we will explore dataset with **more than two** possible classes.



Multi-Class Classifiers

The **Churn_yn.xlsx** dataset used until now, is a binary classification dataset.

In this section we will explore dataset with **more than two** possible **classes**.

Many ML models are already capable of deal with te multi-classification task.
For example:

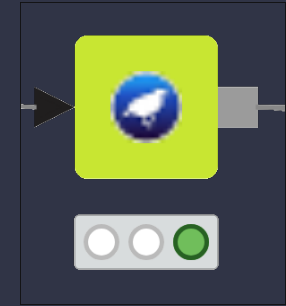
- J48
- Random Forest
- XGboost
- Naive Bayes



MultiClassClassifier Node for Binary Classifiers

Some classifiers, such as **Logistic Regressor**, only support the binary classification task.

MultiClassClassifier

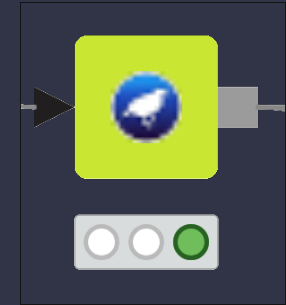


MultiClassClassifier Node for Binary Classifiers

Some classifiers, such as **Logistic Regressor**, only support the binary classification task.

The *MultiClassClassifier* node (provided by weka) allows to overcome this limitation with a **one-vs-all** approach.

MultiClassClassifier

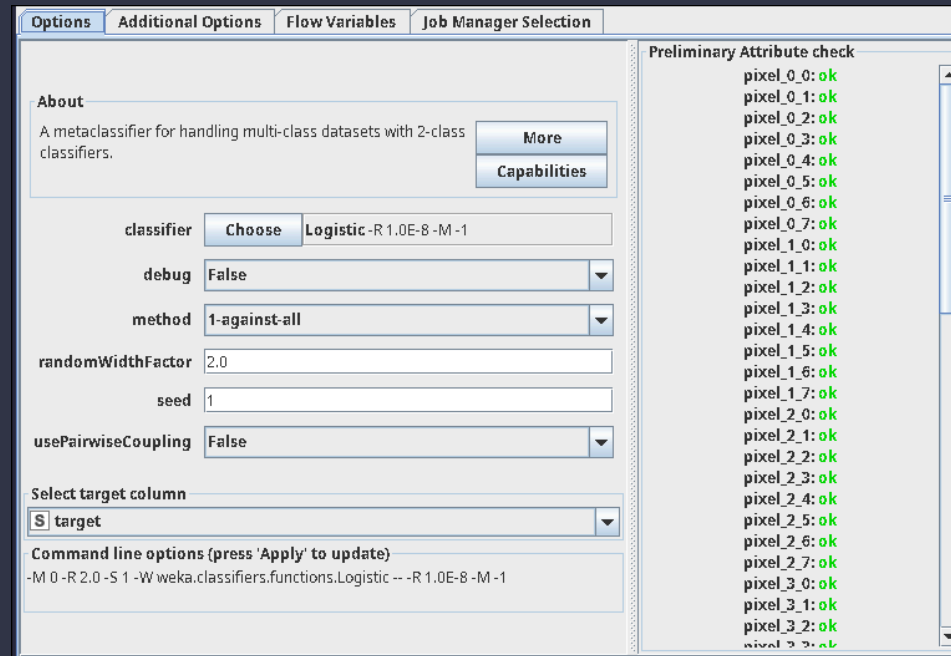
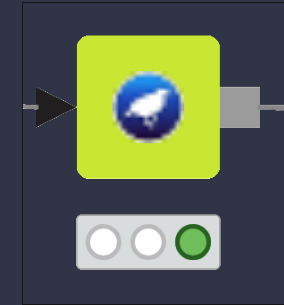


MultiClassClassifier Node for Binary Classifiers

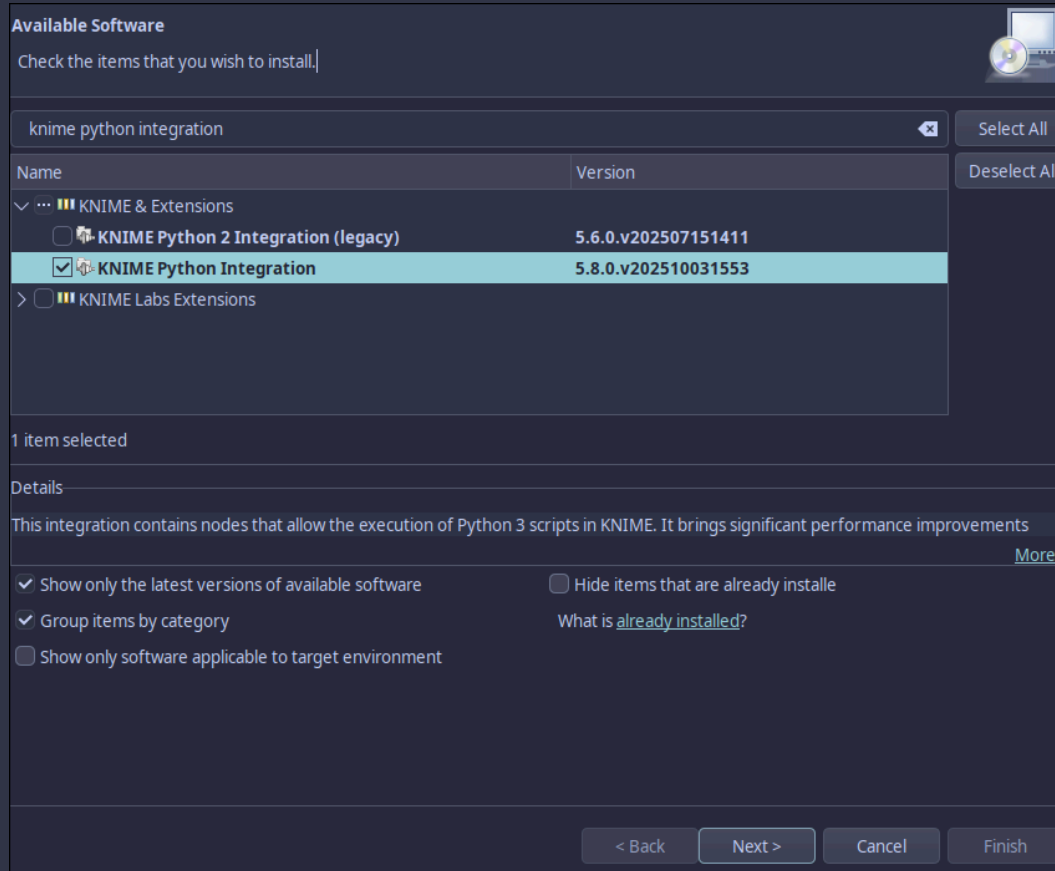
Some classifiers, such as **Logistic Regressor**, only support the binary classification task.

MultiClassClassifier

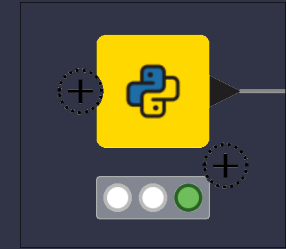
The *MultiClassClassifier* node (provided by weka) allows to overcome this limitation with a **one-vs-all** approach.



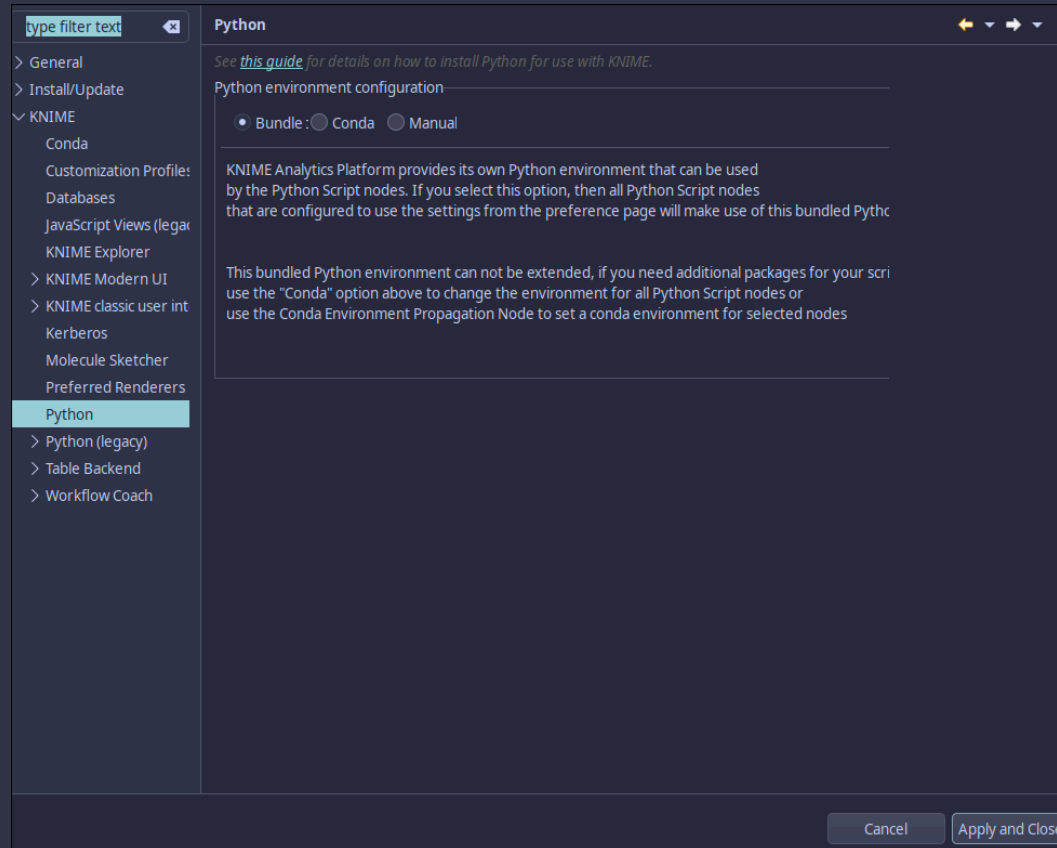
Python Extension Installation



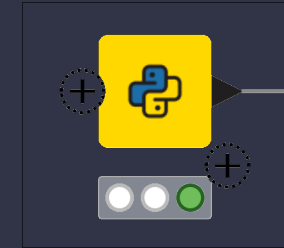
Python Script



Python Extension Configuration

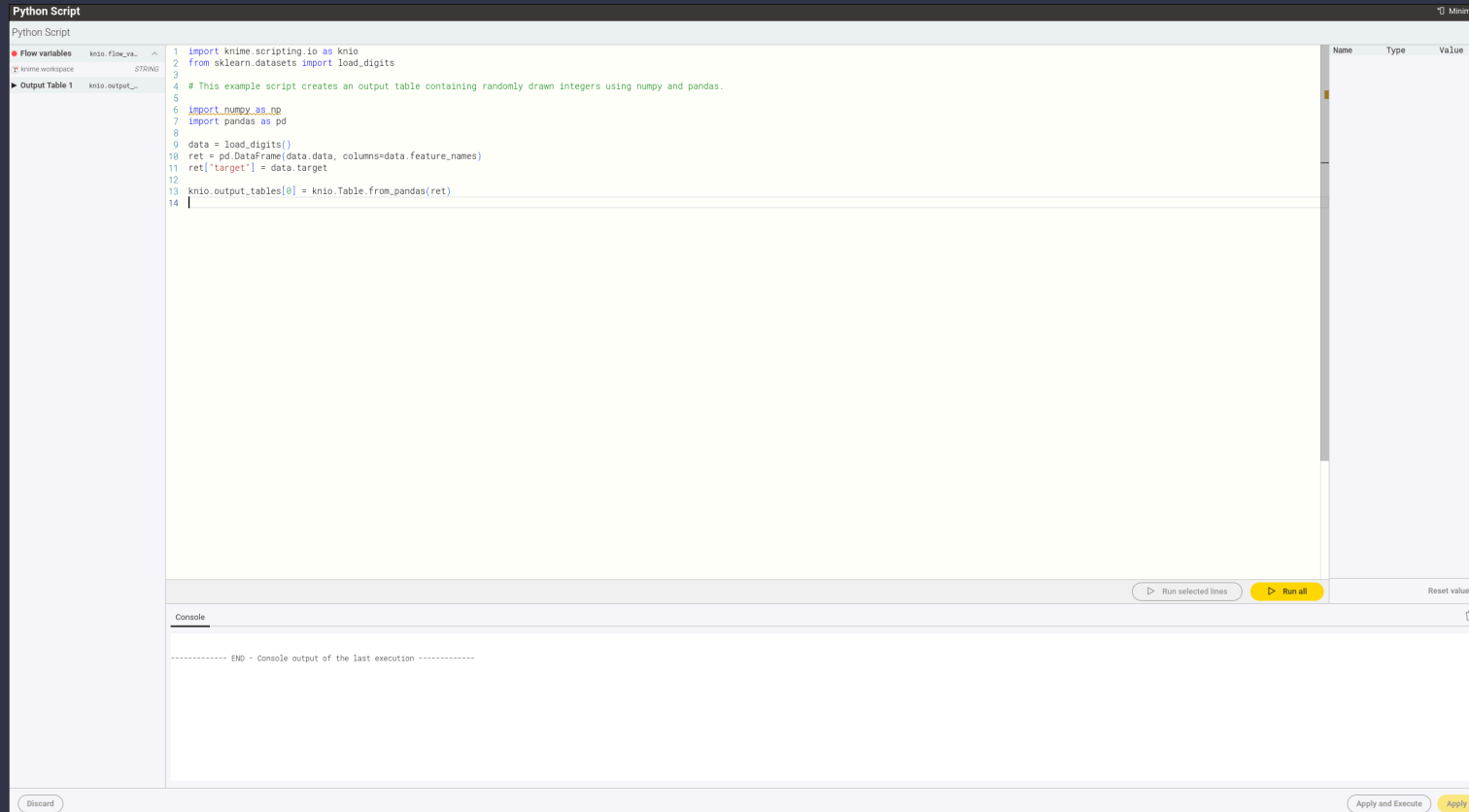


Python Script

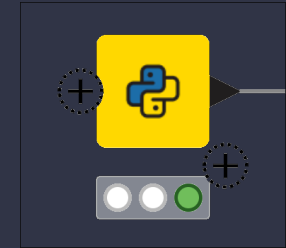


Python Extension

Python Script config



Python Script



Hands on (20 minutes)

- **Import** the **digit** dataset from sklearn using the python script node
- **Convert** the target variable to string
 - *hint: use the table manipulator node*
- Use the nodes **X-partitioner** and **X-aggregator** to cross validate the following models:
 - Logistic Regression
 - *hint: use MultiClassClassifier node*
 - Naive Bayes
 - *hint: use the nodes NaiveBayes Classifier and NaiveBayes predictor*
- **Evaluate** the models using the **Scorer** node.
- **Evaluate** the models with the AUC.
 - *hint: AUC is a binary measure you have to compute one curve for each class.*

